U208: a three-cell fold for the consistency of Peano arithmetic

Five U209 proposals, two machines, one further state deletion

Research prepared for Vladimir Reshetnikov • 29 October 2026

Abstract

The five code-bearing 209-state proposals in the specified ProveIt snapshot represent two rooted, action-labeled transition graphs. Four are the same machine up to state renaming; the Compatibility package contains the other machine. We compare their source layouts, proof scopes, reconstruction programs, and freshly reproduced verification results.

Starting from the Joint/Compatible-Folds table, we delete native state 211 and reuse state 172. Only three retained transition rows change. The proof uses information from the preceding execution: state 211 scanning zero has a one immediately to its left, whereas state 172 has the predecessor word 10. Four finite paths inspect at most the current cell and its two left neighbors, preserve the whole tape, and simulate each old step by one, three, or seven new steps. This establishes halting equivalence on every common initial tape from state 0, independently of the source program and theory calibration.

The resulting complete binary, single-tape machine has **208 working states and 416 transitions**, including initialization. All 5,084 arbitrary-value primitive contracts were freshly rederived from its literal table, covering every transition row; the complete 48,140-step blank startup is unchanged. The new finite reduction is proved here. The proposed $\mathrm{Con}(\mathsf{U}) \leftrightarrow \mathrm{Con}(\mathsf{PA})$ endpoint retains the incoming mathematical calibration and is not a newly kernel-checked PA, Lean, Rocq, or MM0 theorem.

Contents

1 Result, conventions, and proof boundary **3**

2 What the supplied proposals contain **4**
2.1 Pinned inventory . . . 4
2.2 How equality and difference were established . . . 4
2.3 The shared earlier savings . . . 5
2.4 Fresh reproduction and runtime comparison . . . 5

3 The new deletion: three changed rows **6**
3.1 Literal patch . . . 6
3.2 Why a history predicate is needed . . . 6

4 Complete proof of the new machine equivalence **6**
4.1 Incoming inventories . . . 6
4.2 Four exhaustive local cases . . . 7
4.3 Ordinary steps and positive progress . . . 8

5 Verification of the delivered U208 table **9**
5.1 Two independently implemented finite checks . . . 9
5.2 Fresh arbitrary-value runtime derivation . . . 9
5.3 Exact initialization and source correspondence . . . 10
5.4 Runtime cost of the new saving . . . 10

6 Logical calibration: what is inherited **11**
6.1 Unary U and its PA interpretation . . . 11
6.2 The machine-facing calculus is a different presentation . . . 11
6.3 Relationship to the supplied ZFC verifications . . . 12

7 Rejected reductions and lessons from the search **12**
7.1 An attractive sharing transformation that fails . . . 12
7.2 Shortening the origin detector also fails directly . . . 13

8 Further research questions **13**

9 Reproduction and archive map **13**

A Literal identities **14**

B Complete dense U208 transition table **15**

1 Result, conventions, and proof boundary

The main deliverable is `machine/u208.tm`; Appendix B prints its complete table. It is obtained from the Joint U209 table by a deterministic edit and a checked dense renaming.

Working states	$0, 1, \ldots, 207$
Separate halt label	208
Alphabet and blank	$\{0, 1\}$, blank 0
Tape	One two-way infinite tape
Initial configuration	State 0, head at 0, all cells 0
Transition format	$q\ b\ w\ D\ r$: write, move, enter r
Movement	Every row moves exactly one cell, left or right
Initialization	Included in the 208 working states
Total rows	416, with no missing read case

Entering the separate halt label halts after the indicated write and move. The companion `machine/u208.native.tm` retains sparse historical state names and uses 254 as its halt label. A native name such as 211 is an identifier, not a state count. All state numbers used in the proof below are *Joint native labels*. The dense table is the submission-ready object.

Let P denote the exact Joint U209 parent and M the new U208 machine. The new result is

$$\mathsf{PA} \vdash \mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Halt}(P, \mathrm{blank}). \tag{1}$$

More generally, the ordinary mathematical proof applies to every common initial tape from state 0. For first-order arithmetization one uses coded finite-support tapes and finite run histories; the blank-tape instance in (1) is immediate. No consistency assumption is used.

The supplied proposals identify the parent's nonhalting with consistency by their retained numerical-calculus, enumeration, and interpretation arguments. Composing that calibration with (1) gives the intended endpoint

$$\mathsf{PA} \vdash \neg\,\mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathsf{U}) \leftrightarrow \mathrm{Con}(\mathsf{PA}). \tag{2}$$

The new fold does not strengthen the verification status of that inherited calibration. This report supplies a complete local reduction proof and executable checks. It does not supply a new end-to-end proof-assistant certificate, prove consistency, or infer nonhalting from a finite run.

Claim	Evidence and scope
Exact 208-state object	Complete native/dense tables, totality, target closure, graph reachability, bijective renaming, deterministic reconstruction.
New $P \leftrightarrow M$ reduction	Complete incoming-history analysis, four local paths, 414 ordinary-row identities, positive-block simulation.
Literal source implementation	Fresh universal symbolic replay of all 5,084 contracts and exact whole-tape blank initialization.
Consistency endpoint	Composition with the incoming source/calculus and U/PA proof translations; not newly formalized here.

2 What the supplied proposals contain

2.1 Pinned inventory

The comparison is frozen at ProveIt commit

`fe8b3a2d1db0033113e986590f79ecba3e5ef500`.

All eight entries in the downloaded directory inventory, including its README, were checked against their Git blob identities. The seven ZIPs are the five code-bearing U209 proposals, the extra Joint PDF-only archive, and `pa_unary_power_cover.zip`. Exact sizes, download URLs, Git object identities, and SHA-256 hashes are retained in `provenance/`. The five input construction reports are references [1, 2, 3, 4, 5].

Proposal	Class	Depth	Distinctive presentation
Plain U209	A	42	Thirty rotations from U212; an additional obstruction to removing a dominated propositional axiom row.
Composed	A	42	Thirty rotations with a label permutation; continuation transport and calibration exposition.
Compatible Folds	A	42	Staged emitter, combined, and prefold tables; separate local and source audits.
Joint	A	42	Direct allocation of an 18-selector region; compact deterministic recipe and complete retained lineage.
Compatibility	B	41	A different allocation across a shared helper; Research U211 to U210 to U209.

The four Class A packages encode the *same literal machine* after renaming. Compatible Folds and Joint have byte-identical native tables, dense tables, and final source maps. Plain and Composed use other native names for a few selectors. The extra `U209_ConPA_Joint_2026-10-29(1).zip` contains only a PDF, which identifies the Plain table by its hashes; it is not a sixth implementation.

2.2 How equality and difference were established

The independent comparison program performs a breadth-first traversal from state 0, always visiting the read-zero edge before the read-one edge. It assigns canonical names on first encounter and preserves the read, write, move, and halt information. All 209 working states are reachable in each transition graph. This is graph reachability, not a claim that every state is visited on the blank computation.

For each of the ten pairs of packages, the checker also constructs the forced state bijection and verifies every transition equation. Four packages have the same canonical graph; attempted bijections to Compatibility fail with a recorded finite conflict. This establishes two isomorphism classes rather than merely two groups of file hashes.

For example, the nontrivial Joint-to-Plain native renaming is

$$129 \mapsto 217, \quad 204 \mapsto 215, \quad 214 \mapsto 129, \quad 216 \mapsto 204, \quad 217 \mapsto 214, \quad 215 \mapsto 216.$$

Every other label is fixed. Consequently the new helper 217 in this report is called 214 in Plain. Copying the numeric patch across packages without this renaming would produce a different, unjustified machine.

All five source programs have the same ordered 3,909 primitives and operands and the same 5,083 guarded ordinal control edges. Their source maps have 197 distinct syntax subtrees and maximum zero-run length six. Classes A and B differ in precisely 720 instruction addresses, the ordinal interval $[3016, 3736)$. Equality of the ordinal graph was checked independently by binary-address increment, ancestor stripping, and zero-padded target decoding, including the absence of stray ones beyond the decoded leaf.

2.3 The shared earlier savings

The proposals all inherit three savings from the same U212 lineage. Class A deletes native states $180, 181, 218$ by the identifications

$$180 \mapsto 163, \qquad 181 \mapsto 220, \qquad 218 \mapsto 157.$$

The first two folds use a bit written by the incoming transition to distinguish two old roles. The third substitutes an existing three-step behavior for a decoder entered with a predecessor zero. Reassociation is needed because the separately proposed U211 patches do not coexist on their original layouts. In Joint labels the retained selector rows include $163/0 \mapsto 0\,L\,211$ and $220/0 \mapsto 0\,L\,129$.

Compatibility reaches a different arrangement. Its final contextual fold is $180 \mapsto 216$, and the corresponding changed row is $216/0 \mapsto 0\,L\,211$. A backward call into helper 129 reads a known zero on the return leg. That call uses only rows 158/0 and 204/0 after the tag test, so row 204/1 remains available for changing the source layout. The eight-factor reassociation exploits exactly this restricted interface.

These observations distinguish useful mechanisms from presentation changes. Renaming Class A or giving another sequence of rotations to it cannot reduce its state count. A new deletion is needed. The present construction deletes the already shared helper 211 itself and compensates for its two entry directions using a stronger execution-history invariant.

2.4 Fresh reproduction and runtime comparison

All five supplied quick reproduction commands passed in isolated working copies. One representative of each isomorphism class was then replayed fully. No saved success flag was substituted for these executions.

Fresh full verification	Joint, Class A	Compatibility, Class B
Arbitrary-value primitive contracts	5,084	5,084
Literal rows covered	418/418	418/418
Symbolic inference applications	1,250,049	1,248,748
Exact blank startup steps	48,140	48,140
Fresh cases equal packaged records	Yes	Yes

The two classes do not have a uniform runtime ordering. Their per-primitive affine duration expressions have equal nonconstant coefficients, but different constants. For the same logical registers and gap/spare parameters, Compatibility is faster on 329 contracts, slower on 466, and equal on 4,289. The differences range from -6 to $+70$ literal steps. The histogram and case comparison are retained in `evidence/duration_comparison.json`. A smaller maximum source depth or fewer symbolic inference applications is therefore not an overall speed claim. The actual frequency of those primitive cases is not determined by this comparison.

Joint is selected as the main baseline because it is byte-identical to Compatible Folds, its deterministic construction is compact, and it retains the complete nested calibration material. This is an auditability choice, not a statement that Class A is faster than Class B.

3 The new deletion: three changed rows

3.1 Literal patch

Define the projection

$$\rho(211) = 172, \qquad \rho(q) = q \quad (q \neq 211).$$

Delete the two rows whose source state is 211. Redirect every retained target through ρ. Finally replace the read-zero row at 172. The entire change to the retained native rows is

State	Read	Parent action	U208 action
163	0	$0\,L\,211$	$0\,L\,172$
163	1	$1\,R\,211$	$1\,R\,172$
172	0	$0\,L\,67$	$0\,L\,217$

The removed rows are $211/0 \mapsto 0\,R\,157$ and $211/1 \mapsto 1\,R\,184$. The retained row $172/1 \mapsto 1\,R\,184$ already agrees with the latter. No source instruction, source address, opcode, axiom formula, numeric convention, initialization routine, or tape alphabet is changed.

The final table has 208 working states. Its dense-state map is the increasing bijection from the retained native names to $0, \ldots, 207$, extended by $254 \mapsto 208$ at the halt. Both directions of this conjugacy are checked against every row.

3.2 Why a history predicate is needed

State 211 has two entrances from state 163. One moves right and writes one; the other moves left and writes zero. Thus a test that requires every entrance to a state to write the same predecessor tag cannot justify this fold. The useful invariant is conditional on the *currently scanned* symbol, and its proof looks one step farther into the incoming history.

State 172 also moves in a different direction on its zero branch than state 211. A direct row identification is invalid. The replacement must inspect the predecessor, and in one case the predecessor's predecessor, before it can recover the correct old action. The tape already contains the required information; no new symbol or state is needed.

4 Complete proof of the new machine equivalence

4.1 Incoming inventories

For a configuration $(q, h, \mathcal{T})$, h is the head coordinate and $\mathcal{T}(i) \in \{0, 1\}$ is the bit in cell i. The following are the complete incoming-row inventories in the exact parent table. A source/read entry lists the row that enters the indicated target.

Target	Incoming source/read	Action entering target
211	163/0; 163/1	$0\,L\,211$; $1\,R\,211$
172	116/0	$0\,R\,172$
116	215/1	$1\,R\,116$
163	132/1; 199/1; 217/0	$1\,R\,163$; $1\,R\,163$; $0\,R\,163$
217	216/1	$1\,R\,217$

All these states differ from the initial state 0. The inventories are computed from all 418 parent rows, not inferred from an observed blank run.

Lemma 4.1 (Conditional predecessor at 211)**.** *In every parent run starting in state 0, if state 211 scans zero at head position h, then $\mathcal{T}(h-1)=1$.*

Proof. The last transition is from 163. If it is 163/1, it writes one at $h-1$ and moves right to h, proving the assertion immediately.

Otherwise the last transition is 163/0, made at $h+1$. It writes zero there and moves left to h. Every entrance to 163 moves right, so the preceding transition entered 163 from h and wrote the bit now scanned by 211. That bit is zero. The complete inventory says that the only such zero-writing entrance is 217/0. Every entrance to 217 is 216/1, which previously wrote one at $h-1$ and moved right to h. The intervening two steps write only at h and $h+1$. The one at $h-1$ is unchanged.

The history cannot terminate prematurely at initialization: 211, 163, and 217 are not state 0. Thus all required preceding transitions exist. □

Lemma 4.2 (Two predecessor cells at 172)**.** *In every parent run starting in state 0, a visit to state 172 at position h satisfies $\mathcal{T}(h-2)=1$ and $\mathcal{T}(h-1)=0$.*

Proof. The unique entrance is 116/0, which writes zero at $h-1$ and moves right. The unique entrance to 116 is 215/1, which previously wrote one at $h-2$ and moved right. The next transition leaves that cell untouched. Neither 172 nor 116 is the initial state, so the two-step history exists. □

These lemmas concern *parent* configurations at simulation boundaries. They need not hold at every intermediate visit to the reused state 172 inside a child block. Confusing those two roles would make a circular invariant argument. The following literal paths handle the intermediate visits explicitly.

4.2 Four exhaustive local cases

The unchanged helper rows used by the new blocks are

$$217/0 \mapsto 0\,R\,163, \qquad 217/1 \mapsto 1\,R\,215,$$
$$215/0 \mapsto 0\,R\,157, \qquad 157/0 \mapsto 0\,L\,67.$$

The child has $163/0 \mapsto 0\,L\,172$ and $172/0 \mapsto 0\,L\,217$ after the patch. The shared read-one row is $172/1 \mapsto 1\,R\,184$.

Normalize the old head coordinate to zero. The table lists each child path as (state, head offset), including its endpoint.

Old state	Read	Known cells	Child path	Steps
211	0	$\mathcal{T}(-1) = 1$	$(172, 0) \to (217, -1) \to (215, 0) \to (157, 1)$	3
211	1	–	$(172, 0) \to (184, 1)$	1
172	0	$\mathcal{T}(-2)\mathcal{T}(-1) = 10$	$(172, 0) \to (217, -1) \to (163, 0) \to (172, -1) \to (217, -2) \to (215, -1) \to (157, 0) \to (67, -1)$	7
172	1	–	$(172, 0) \to (184, 1)$	1

The currently scanned bit is the value in the Read column. Every transition on these paths writes back the bit it reads. The endpoints are exactly the parent one-step endpoints, with the state projected by ρ: $(157, 1)$, $(184, 1)$, $(67, -1)$, and $(184, 1)$ respectively. No internal state is the halt. The longest path reads only offsets $-2, -1, 0$. Every other cell is unread and unwritten; this is a whole-tape identity with an arbitrary exterior, not a test of finitely many chosen tapes.

The seven-step case revisits state 172 one cell left. At that intermediate visit the predecessor is one, so the three-step route used for the first case is followed. Its termination is established by the displayed path; there is no appeal to an unproved recursive simulation claim.

4.3 Ordinary steps and positive progress

For the 414 parent rows whose source state is neither 211 nor 172, the tables satisfy the exact quotient equation

$$\delta_M(\rho(q), b) = (w, D, \rho(r)) \quad \text{whenever} \quad \delta_P(q, b) = (w, D, r). \tag{3}$$

The two redirected rows of 163 satisfy this equation. The four exceptional rows are precisely the four cases above.

Theorem 4.3 (All-tape positive-block simulation)**.** *Start P and M in state 0 with the same tape and head coordinate. At matched boundaries the head and entire tape agree, and the child state is ρ of the parent state. Every parent step is simulated by a positive child block of length at most seven. If a parent prefix has n steps, its matching child prefix has exactly*

$$n' = n + 2N_{211,0}(n) + 6N_{172,0}(n), \tag{4}$$

where $N_{q,b}(n)$ counts parent steps in that prefix executed at state q while reading b. In particular, $n \le n' \le 7n$. The two machines have the same halting behavior from every such common initial configuration.

Proof. Initially $\rho(0) = 0$. Induct on parent steps. At an ordinary state, (3) supplies one child step with the required endpoint. At state 211 or 172, the two predecessor lemmas supply exactly the bits required by the applicable local case. That case supplies the matching positive block and preserves the entire tape. Concatenation proves the boundary correspondence. Counting the lengths $3, 1, 7, 1$ gives (4) and its bounds.

If the parent halts, the concatenated child blocks reach the same halt label, which is fixed by ρ. For the reverse implication, suppose the child halts after at most t steps but the parent has not halted within t steps. Simulating that parent prefix yields a child prefix of at least t steps containing no halt: ordinary steps cannot halt unless the parent does, and no exceptional block has an internal halt. Determinism makes this the actual child prefix, contradicting the assumed child halt. Thus a child halt within t steps implies a parent halt within t steps. This establishes both directions using finite prefixes only. □

Corollary 4.4 (PA-formalizable reduction)**.** *For blank input,* (1) *holds. The finite-horizon implications can be expressed in PA as*

$$\mathrm{Halt}_{\le n}(P) \implies \mathrm{Halt}_{\le 7n}(M), \qquad \mathrm{Halt}_{\le n}(M) \implies \mathrm{Halt}_{\le n}(P).$$

Proof. The tables and all local cases are fixed finite objects. Histories, tape updates, and runs have primitive-recursive codes. The predecessor lemmas inspect at most three transitions; the global proof is induction on a finite coded run length. The same proof therefore gives the stated PA derivations. This explains formalizability; it is not an exported PA proof object or a claim that the Python programs are proof-assistant kernels. □

5 Verification of the delivered U208 table

5.1 Two independently implemented finite checks

`code/check_fold.py` checks the complete incoming inventories, the required helper rows, all four paths, all 414 ordinary quotient equations, totality, graph reachability, and the one-state deletion. Its conditions are explicit exceptions and remain active with Python `-O`. It rejects eight damaged constructions, including wrong direction, wrong write, wrong helper target, and damaged predecessor conditions.

The separate `code/check_fold_independent.py` imports neither the builder nor the discovery checker, source parser, runtime engine, or saved verdict. It enumerates all bounded incoming histories and solves their literal read/write constraints. For state 211 scanning zero there are six length-three state histories: two are inconsistent with the scanned zero, and all four remaining histories force the left bit to one. The unique length-two history into 172 forces the predecessor word 10. The checker also excludes entries too close to initialization to have the histories used by the proof. It independently checks the exact child paths and full table projection.

The negative controls are error-detection checks, not an additional theorem. The theorem is the invariant and finite-block argument above. The independent history analysis reduces the risk that a hand-selected predecessor census silently omitted an entrance.

5.2 Fresh arbitrary-value runtime derivation

The source remains byte-identical to Joint's. Nevertheless, the literal runtime contracts were freshly derived from the final U208 table. The two inherited symbolic engine modules are copied unchanged and checked against their frozen SHA-256 identities before execution.

Fresh U208 verification item	Result
Working states / transitions	208 / 416
Source primitive occurrences	3,909
Guarded primitive contracts	5,084
Literal transition coverage	416 / 416
Symbolic inference applications	1,251,865
Minimum affine duration constant	5
Duration coefficients nonnegative	All
Primary arithmetic registers	Nine arbitrary naturals
Gap parameter	Arbitrary natural
Spare registers	Arbitrary nonempty finite list

Each decrement primitive has separate zero and positive cases. The engine uses literal steps, induction over same-state unary scans, four rederived arbitrary-block traversal laws extended by list induction, and explicit word normalization. It refuses to inspect a possibly empty symbolic run without a positivity justification. Every case has positive affine duration; every nonhalting primitive reaches its expected whole-tape final frame, and the halt primitive reaches the literal halt label. The complete per-case records are compressed in `evidence/runtime_cases.json.gz`.

This is stronger than a bounded numerical test: register values, gap, and spare-list length are symbolic. It is still executable mathematical induction implemented in ordinary Python. The inference implementation has not been checked by a proof-assistant kernel in this release. The direct fold theorem does not depend on trusting that engine; the replay provides an additional check that the emitted final table realizes the inherited primitive interface.

5.3 Exact initialization and source correspondence

The parent and child blank runs agree transition by transition for the first 48,140 steps. They finish at native state 84 with the head at -2, header at 54, source ordinal 136, and source address `001100110101010`. The logical register tuple is

$$(1, 0, 13, 0, 0, 1, 0, 0, 1),$$

with 43 spare registers of value one. The entire finite-support tape has exactly 120 one-cells; all other cells are zero. The new patch is not visited during this startup. Both the full support and the complete trace digest are compared, rather than only a few visible registers.

The inherited startup lands inside a protected axiom-builder continuation, not at a clean abstract proof-search root. Its correctness depends on the retained source/calculus startup argument. The new fold preserves that continuation exactly; it does not reinterpret the initial cached value one as a theorem. This distinction remains part of the logical calibration.

The exact trace SHA-256 is

`41e9a165abae389c3b182ebbf8d8fd1c260d3d36d4f7ebb6d60e235bac40f19a`.

5.4 Runtime cost of the new saving

Equation (4) is an exact comparison with the chosen U209 parent on any corresponding run prefix. This is a state saving, with an at-most sevenfold step overhead. It is not a claimed speed improvement.

The bound can be sharpened for a particular execution by counting its two exceptional zero-read cases, but no distributional claim about the consistency search is needed for correctness.

The source addresses and control graph do not change in this reduction, so its all-tape scope is stronger than the earlier reassociation stages. It does not retroactively make those earlier source transformations all-tape simulations.

6 Logical calibration: what is inherited

6.1 Unary U and its PA interpretation

The proposed theory U is classical equality-free first-order logic over a nonempty domain with one binary relation, written membership. Define

$$x \subseteq a \quad :\Longleftrightarrow \quad \forall t\,(t \in x \to t \in a).$$

For every formula $\varphi(x)$ with no other free variable, its sole nonlogical schema is

$$\forall a\,\exists b\,[\forall x\,(x \in b \to \varphi(x)) \;\wedge\; \forall x\,(x \subseteq a \to (\varphi(x) \to x \in b))]\,. \tag{5}$$

Bound variables are freshened. The restriction concerns free matrix parameters; it is not a restriction to one-variable logic. The schema provides a weak cover and does not assume extensionality or an exact power filter for every ambient source.

The unary report [6] gives interpretations in both directions and uniform finite-refutation transformations. In the upper interpretation, membership is binary-bit membership in a PA model. The subset condition bounds the candidate elements, and formula-specific finite comprehension is proved by PA induction. This also proves irreflexivity. The proof does not evaluate a general truth predicate.

The lower interpretation uses numerical representatives modulo coextensionality, with saturation providing the needed membership congruence. Its construction order is consequential: local predecessor facts first give induction without total successors; comparison and least numerical failures then give bounded numerical comprehension with the parameters bound inside legal unary matrices. Total successors, numerical powers, saturated ordered-pair codes, and canonical bounded histories follow. Addition and multiplication are defined by those histories, and translated PA induction uses the earlier induction theorem.

Our reading found no concrete illicit free parameter or circular successor assumption in those examined steps. The package supplies ordinary mathematical proof templates, not an implemented general translator for arbitrary PA proofs or a kernel certificate. We retain that scope rather than presenting finite diagnostics as a new proof of equiconsistency.

6.2 The machine-facing calculus is a different presentation

The U209 machine does not implement a unary-free-variable validator. Its inherited numerical calculus is calibrated to unrestricted weak covering W, allowing additional matrix parameters, together with global irreflexivity. Unary U embeds into this theory; the binary-membership interpretation in PA validates the additional instances and irreflexivity. These interpretations bridge the consistency presentations.

The native numerical coding uses dual Cantor pairing with constructor tags $4P$, $4P+1$, $4P+2$, and $4P+3$. Code zero is a truth alias; code one is diagonal membership used as falsity under irreflexivity.

The retained calibration covers the logical rows, the dominated propositional row, modus ponens, generalization, the gated covering/vacuity bundle, and the exceptional small-code and variable-collision cases. Completeness also requires compiling proofs above arbitrary previous stack contents and cofinal successful enumeration.

This is *not* the separate positive dyadic theorem stream described in the unary-theory ZIP. That report's positive stream is another calibrated interface; its existence does not mean it has been compiled into the delivered table. The new U208 reduction preserves the actual U209 source and hence preserves its existing calibration obligations exactly.

The complete older U222 mathematical material is present in the retained Joint archive through its selected U212, U213, U214, and U218 ancestry. The included logical reference sources have unbundled imports and should not be described as a runnable new U208 Rocq project. Original warnings superseded by the exceptional-code correction must be read together with that correction, as the incoming reports explain.

6.3 Relationship to the supplied ZFC verifications

The m254 project supplies external Coq endpoints for its literal table, calculus, and ZF/ZFC target [7]. ProveIt's K215 development supplies external Lean and Rocq endpoints for its exact K215 table [8]. The separate k215-pa repository supplies the internal PA theorem in Metamath Zero [9]. These are relevant examples of end-to-end verification architecture, but none certifies the different U208 table or its PA calibration. Those large projects were read for scope and were not rebuilt during this investigation.

7 Rejected reductions and lessons from the search

7.1 An attractive sharing transformation that fails

A structural search suggested reassociating

$$((98, 12), 150) \longmapsto (98, (12, 150)), \qquad ((166, 12), 150) \longmapsto (166, (12, 150)).$$

At the table level these are rotations of $(155, 85)$ and $(52, 154)$. Afterward states 85 and 154 have identical rows, apparently saving one state by sharing the suffix $(12, 150)$. The final identical-row quotient is valid; the preceding rotation is not justified.

The reason is hidden reuse through backward execution. A purely rightward source traversal sees three occurrences of 155. Full literal prefix analysis finds five occurrences of 155 and ten bindings to 85, including five not under those 155 roots. One such path is

$$76 \to 152 \to 159 \to 118 \to 155 \to 85.$$

It revisits already consumed bits. The exact source prefix `0011001000001`, with a preceding one, exposes the failure: the old machine completes at state 42 and offset 13 without changing the tape; the rotated machine reaches state 41 at offset 12 and overwrites a one with zero. The counterexample is retained in `research/rejected_rotation.json`. No machine from that failed route is included as a candidate.

This is a useful limitation on source-DAG optimization. Literal equality of two rows after a source edit proves the quotient stage only. One must also validate all physical entries to every repurposed decoder, including entries that reread earlier address bits.

7.2 Shortening the origin detector also fails directly

Another test attempted to delete native state 34 from the seven-zero origin detector by redirecting 23/0 directly to 75. All source-address six-zero runs are terminal after trimming the suffix, which initially suggested a six-zero detector might suffice. The actual runtime uses temporary markers, so a static trimmed-address statistic is insufficient. The shortened table already fails the first primitive's zero-decrement symbolic contract at source address `000000`. This exploratory table is not delivered.

Both failed ideas were screened against the literal runtime before a state saving was accepted. The delivered 208-state result is solely the three-cell history fold proved in Section 4.

8 Further research questions

1. **Can bounded-history certificates become a systematic optimizer?** The successful predicate couples the current read bit with earlier writes; it is missed by a uniform one-cell incoming-tag test. Enumerating feasible short transition histories, solving bit constraints, and synthesizing local replacement paths provides a finite search space with independently checkable certificates. The useful next objective is one more sound deletion, not a claim that the current search has exhausted all 207-state possibilities.
2. **Can the generic fold theorem be formalized once?** A proof-assistant library could accept a state projection, finite incoming history covers, ordinary-row equations, and finite positive blocks, then produce the blank-halting equivalence for a literal table. This would make the incremental U209-to-U208 proof small. It would not by itself formalize the separate numerical-calculus and U/PA interpretation layers.
3. **Can source sharing account for every physical decoder entry?** The rejected rotation suggests enriching each source node with its complete literal prefix interface, including known previously read bits and backward entries. A transformation should transport those interfaces before counting a shared suffix as a state saving.
4. **Do the two decoder allocations offer different next savings?** Class B has a different 720-primitive address region and maximum depth 41. The present history-fold mechanism also checks locally in its native layout, with the appropriate helper names. The two layouts should remain distinct search inputs rather than being counted as five independent designs.
5. **Which inherited calibration step should receive the next kernel proof?** Priority candidates are the numerical-calculus refutation bridge, protected nonzero startup, and the unary U/PA proof translations. These are precisely where an incremental table certificate cannot replace an end-to-end proof.

No lower bound or global optimality assertion follows from this investigation. The achieved result is an explicit, reproducible improvement from 209 to 208 working states with a small new halting-equivalence proof.

9 Reproduction and archive map

Use Python 3.10 or later with its standard library and enabled assertions. The default command is offline and writes verification outputs to a temporary directory:

```
python3 -B reproduce.py
```

It verifies release hashes, reconstructs both literal tables, checks the native/dense map, runs both finite fold checkers, reruns the comparison, checks exact startup, and freshly derives all 5,084 runtime contracts. The runtime phase took about 151 seconds in the preparation environment; that elapsed time is environment-dependent, not a property of the machine.

```
python3 -B reproduce.py --quick
```

The quick option explicitly skips the full symbolic runtime derivation. It still checks the complete new finite fold theorem and startup, but is not a substitute for the full replay when validating the primitive engine. The local checker is additionally exercised with Python optimization enabled; the inherited symbolic runtime refuses that mode because it uses assertions.

Path	Contents
`report.tex`, `report.pdf`	This report; editable source and compiled output.
`paper/`	Generated complete dense transition table.
`machine/`	Final native/dense U208 tables, checked state map, unchanged source.
`baseline/`	Exact Joint U209 native parent and inherited runtime-label map.
`code/`	Deterministic builder, two finite checkers, comparison, startup, and runtime replay.
`comparison/`	Exact table/source snapshots for all five code packets, with their README and notices.
`evidence/`	Fresh receipts, full runtime cases, parent replay logs, and comparison witnesses.
`reference/`	Unchanged Joint U209 ZIP with its complete lineage, and the exact unary-theory ZIP.
`research/`	Rejected-rotation counterexample and bounded search scope.
`provenance/`, `licenses/`	Pinned input identities and retained attribution.
`SHA256SUMS`	Hashes of every other delivered file.

The PDF can be rebuilt with two runs of `pdflatex report.tex` in a working copy. PDF bytes may differ with TeX versions and metadata; the literal machine and source bytes are fixed.

A Literal identities

Parent Joint U209 native table

`d0a155d3c683789f3bf5327c0cfb2a0cfd78076af8f1654d4ca426ccbb604d16`

New U208 native table

`527c2a6065ba7c9adb09294eed2b532810d812e46df4b5b912bae99ae6fb3a3a`

New U208 dense table

`9f4ea95849c5e1d01fa3f740023092230cd7006eed650b94a2d5da537e25212c`

Unchanged source map

`cddbed7f20e7a8b085b7ca0e76811d07b021522375643b5d1134eec0d2d23164`

B Complete dense U208 transition table

The working labels are $0, \ldots, 207$; H denotes the separate halt label 208. Each entry is write, move, target. This table is generated directly from the delivered dense text table; it is not a schematic description. Two state ranges are printed side by side.

State	Read 0	Read 1	State	Read 0	Read 1
0	1 R 92	1 R 0	104	0 R 21	1 R 120
1	0 L 1	1 L 2	105	0 R 21	0 L 10
2	0 L 3	1 L 2	106	0 R 61	1 R 42
3	0 L 5	1 L 2	107	0 R 121	1 R 8
4	1 R 6	0 L 4	108	0 R 101	1 R 122
5	0 L 9	1 L 2	109	0 L 207	0 L 95
6	0 R 6	0 R 10	110	0 R 124	1 R 104
7	0 R 190	1 R 159	111	0 R 115	1 L 72
8	0 R 14	1 R 13	112	0 R 165	1 R 181
9	0 L 15	1 L 2	113	0 R 160	1 R 184
10	0 L 16	0 R 10	114	0 R 148	1 R 146
11	0 L 26	1 R 128	115	0 R 11	1 R 129
12	0 R 19	1 R 20	116	0 L 131	0 L 1
13	0 L 89	1 L 31	117	0 R 39	1 R 72
14	0 L 11	0 R 22	118	0 R 61	1 R 132
15	0 L 23	1 L 2	119	0 R 61	1 R 133
16	1 L 1	0 L 1	120	0 R 70	0 L 16
17	0 R 24	0 R 32	121	0 R 14	1 R 132
18	0 R 98	1 R 113	122	0 R 21	1 R 134
19	0 R 28	1 R 29	123	0 R 151	1 R 189
20	0 R 30	1 R 31	124	0 R 137	1 L 20
21	0 R 24	0 R 59	125	0 R 130	1 R 180
22	0 R 33	1 R 22	126	0 R 203	1 R 156
23	0 L 34	1 L 2	127	0 R 75	1 R 176
24	0 L 35	0 R 0	128	0 R 75	1 R 205
25	0 R 36	1 R 25	129	0 R 202	1 R 157
26	0 R 37	1 R 38	130	0 R 12	1 R 141
27	0 R 39	0 L 40	131	0 R 30	1 R 69
28	0 R 41	1 R 42	132	0 R 39	1 L 124
29	0 R 43	1 R 44	133	0 R 153	0 L 16
30	0 R 45	1 R 46	134	0 R 154	0 L 16
31	0 R 21	1 R 47	135	0 R 88	1 R 136
32	0 R 48	1 R 32	136	0 R 196	1 R 140
33	0 R 32	1 R 33	137	0 R 24	0 R 48
34	0 L 74	1 L 2	138	0 R 99	1 R 37
35	1 R 35	1 L 1	139	0 R 172	1 R 148
36	0 R 50	1 R 36	140	0 R 96	1 R 159
37	0 R 17	1 R 111	141	0 R 175	1 R 166
38	0 R 13	1 R 52	142	0 R 12	1 R 159
39	0 L 11	0 R 36	143	0 R 88	1 L 166
40	0 L 4	1 L 53	144	0 R 88	1 R 158
41	0 R 24	0 R 25	145	0 L 152	1 L 163
42	0 R 13	0 L 16	146	0 R 84	1 R 88
43	0 R 55	1 R 56	147	0 R 159	1 R 12
44	0 R 57	1 R 58	148	0 R 83	1 R 143
45	0 R 24	0 R 54	149	0 R 144	1 R 155
46	0 R 60	1 R 61	150	0 L 66	1 L 141
47	0 R 62	0 L 16	151	0 R 150	1 R 170
48	0 R 25	1 R 48	152	0 R 159	1 R 114
49	0 L 35	0 R 63	153	0 R 97	1 R 97
50	0 R 16	1 R 50	154	0 R 49	0 R 32

Continued on the next page

State	Read 0	Read 1	State	Read 0	Read 1
51	0 *R* 147	1 *R* 143	155	0 *R* 192	1 *R* 188
52	0 *R* 67	1 *R* 68	156	0 *L* 165	1 *R* 165
53	0 *L* 40	1 *L* 53	157	0 *L* 187	1 *R* 96
54	0 *R* 22	1 *R* 54	158	0 *R* 168	1 *R* 179
55	0 *L* 11	0 *R* 50	159	0 *R* 19	1 *R* 171
56	0 *R* 60	0 *L* 16	160	0 *R* 164	1 *R* 187
57	0 *R* 69	1 *R* 70	161	0 *R* 159	1 *R* 96
58	0 *R* 61	1 *R* 71	162	0 *R* 125	1 *R* 164
59	0 *R* 54	1 *R* 59	163	0 *R* 97	1 *R* 162
60	0 *R* 49	0 *R* 25	164	0 *L* 190	1 *R* 142
61	0 *L* 28	1 *L* 72	165	0 *L* 200	1 *R* 173
62	0 *L* 117	1 *L* 30	166	0 *R* 146	1 *R* 172
63	0 *R* 73	1 *R* 63	167	0 *R* 96	1 *R* 123
64	0 *R* 97	1 *R* 150	168	0 *R* 161	1 *R* 143
65	1 *R* 185	1 *L* 53	169	0 *R* 148	1 *R* 87
66	0 *R* 144	1 *R* 145	170	0 *R* 161	1 *R* 96
67	0 *R* 76	1 *R* *H*	171	0 *R* 181	1 *R* 122
68	0 *L* 196	1 *R* 31	172	0 *L* 193	1 *R* 60
69	0 *R* 41	1 *R* 62	173	0 *R* 145	1 *R* 88
70	0 *L* 89	1 *R* 78	174	0 *R* 152	1 *R* 201
71	0 *R* 55	1 *L* 79	175	0 *R* 140	1 *R* 84
72	0 *R* 80	1 *R* 81	176	0 *R* 175	1 *R* 75
73	1 *R* 0	1 *R* 63	177	0 *R* 126	1 *R* 169
74	0 *R* 74	1 *R* 82	178	0 *R* 138	1 *R* 116
75	0 *L* 145	1 *L* 141	179	0 *R* 97	1 *R* 51
76	0 *R* 85	1 *R* 86	180	0 *R* 168	1 *R* 51
77	0 *R* 20	1 *R* 38	181	0 *R* 17	1 *L* 131
78	0 *R* 89	1 *L* 132	182	0 *R* 143	1 *R* 149
79	0 *L* 108	1 *R* 56	183	0 *R* 76	1 *R* 116
80	0 *R* 49	0 *R* 33	184	0 *R* 106	1 *R* 124
81	0 *R* 91	0 *L* 42	185	1 *L* 40	1 *L* 109
82	0 *R* 178	1 *R* 77	186	0 *R* 175	1 *R* 156
83	0 *R* 96	1 *R* 12	187	0 *R* 182	1 *R* 183
84	0 *R* 88	1 *R* 97	188	0 *R* 66	1 *R* 204
85	0 *R* 206	1 *R* 21	189	0 *R* 140	1 *R* 197
86	0 *R* 116	1 *R* 100	190	0 *R* 84	1 *R* 191
87	0 *R* 174	1 *R* 167	191	0 *R* 157	1 *R* 186
88	0 *R* 102	1 *R* 103	192	0 *R* 64	1 *R* 7
89	0 *R* 80	1 *R* 79	193	0 *R* 150	1 *R* 127
90	0 *R* 101	1 *R* 195	194	0 *R* 18	1 *R* 94
91	0 *R* 105	1 *R* 106	195	0 *R* 104	1 *R* 96
92	0 *R* 65	0 *R* 0	196	0 *R* 90	1 *R* 88
93	0 *R* 27	1 *R* 107	197	0 *R* 139	1 *R* 199
94	0 *R* 183	0 *L* 207	198	0 *R* 150	1 *R* 112
95	1 *L* 109	1 *L* 95	199	0 *R* 201	1 *R* 200
96	0 *R* 19	1 *R* 110	200	0 *R* 156	1 *R* 198
97	0 *R* 60	1 *R* 60	201	0 *L* 123	1 *R* 168
98	0 *R* 68	1 *R* 135	202	0 *R* 196	1 *R* 28
99	0 *R* 93	1 *R* 108	203	0 *R* 163	1 *R* 193
100	0 *R* 105	0 *L* 207	204	0 *R* 146	1 *R* 177
101	0 *R* 117	1 *R* 118	205	0 *R* 161	1 *R* 194
102	0 *R* 71	1 *R* 58	206	0 *R* 45	1 *R* 100
103	0 *R* 57	1 *R* 119	207	0 *L* 4	0 *L* 207

References

[1] ProveIt, *U209 ConPA*, 29 October 2026. https://github.com/VladimirReshetnikov/ProveIt/blob/fe8b3a2d1db0033113e986590f79ecba3e5ef500/docs/incoming/U209_ConPA_2026-10-29.zip.

[2] ProveIt, *U209 ConPA Composed*, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/fe8b3a2d1db0033113e986590f79ecba3e5ef500/docs/incoming/U209_ConPA_Composed_2026-10-29.zip.

[3] ProveIt, *U209 ConPA Compatible Folds*, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/fe8b3a2d1db0033113e986590f79ecba3e5ef500/docs/incoming/U209_ConPA_Compatible_Folds_2026-10-29.zip.

[4] ProveIt, *U209 ConPA Joint*, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/fe8b3a2d1db0033113e986590f79ecba3e5ef500/docs/incoming/U209_ConPA_Joint_2026-10-29.zip.

[5] ProveIt, *U209 ConPA Compatibility*, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/fe8b3a2d1db0033113e986590f79ecba3e5ef500/docs/incoming/U209_ConPA_Compatibility_2026-10-29.zip.

[6] ProveIt, *Unary power covering at PA strength*, 27 October 2026, archived at the same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/fe8b3a2d1db0033113e986590f79ecba3e5ef500/docs/incoming/pa_unary_power_cover.zip.

[7] Jeff's busy-beaver project, *A verified 254-state Turing machine that halts iff ZFC is inconsistent*, README, accessed 29 October 2026. https://github.com/jeff87654/busy-beaver/tree/zf-254/ZF/m254.

[8] ProveIt, *Formal correctness of the exact 215-state machine*, pinned README. https://github.com/VladimirReshetnikov/ProveIt/tree/fe8b3a2d1db0033113e986590f79ecba3e5ef500/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215.

[9] Elliot Glazer, *K215 halts iff ZFC is inconsistent (Metamath Zero, PA)*, README, accessed 29 October 2026. https://github.com/elliotglazer/k215-pa.